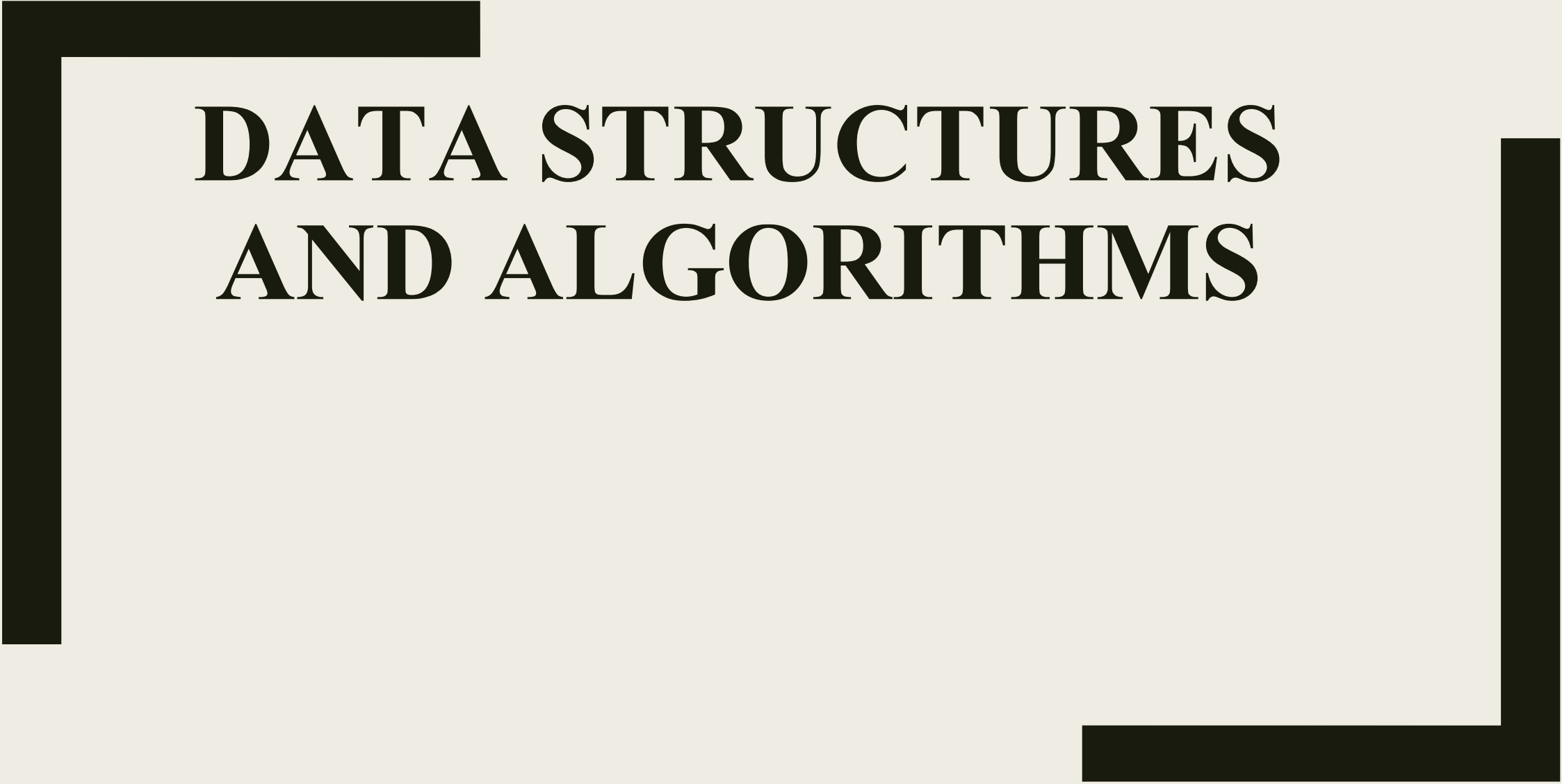


A thick black L-shaped frame is positioned on the left and bottom edges of the slide, framing the title text.

DATA STRUCTURES AND ALGORITHMS

Outline

- Comparing algorithms
- Theoretical analysis of running time
- Asymptotic analysis of running time
- Big O notation

Comparing Algorithms

- Given 2 or more algorithms to solve the same problem, how do we select the best one?
- Some criteria for selecting an algorithm
 - 1) Is it easy to implement, understand, modify?
 - 2) How long does it take to run it to completion?
 - 3) How much of computer memory does it use?
- Software engineering is primarily concerned with the first criteria
- In this course we are interested in the second and third criteria

Comparing Algorithms

- Time complexity
 - The amount of time algorithm needs to run to completion
- Space complexity
 - The amount of memory algorithm needs to run
- Occasionally will look at space complexity, but mostly interested in time complexity in this course
 - *Thus the better algorithm is the one which runs faster (has smaller time complexity)*

How to Calculate Running time

- Most algorithms transform input objects into output objects

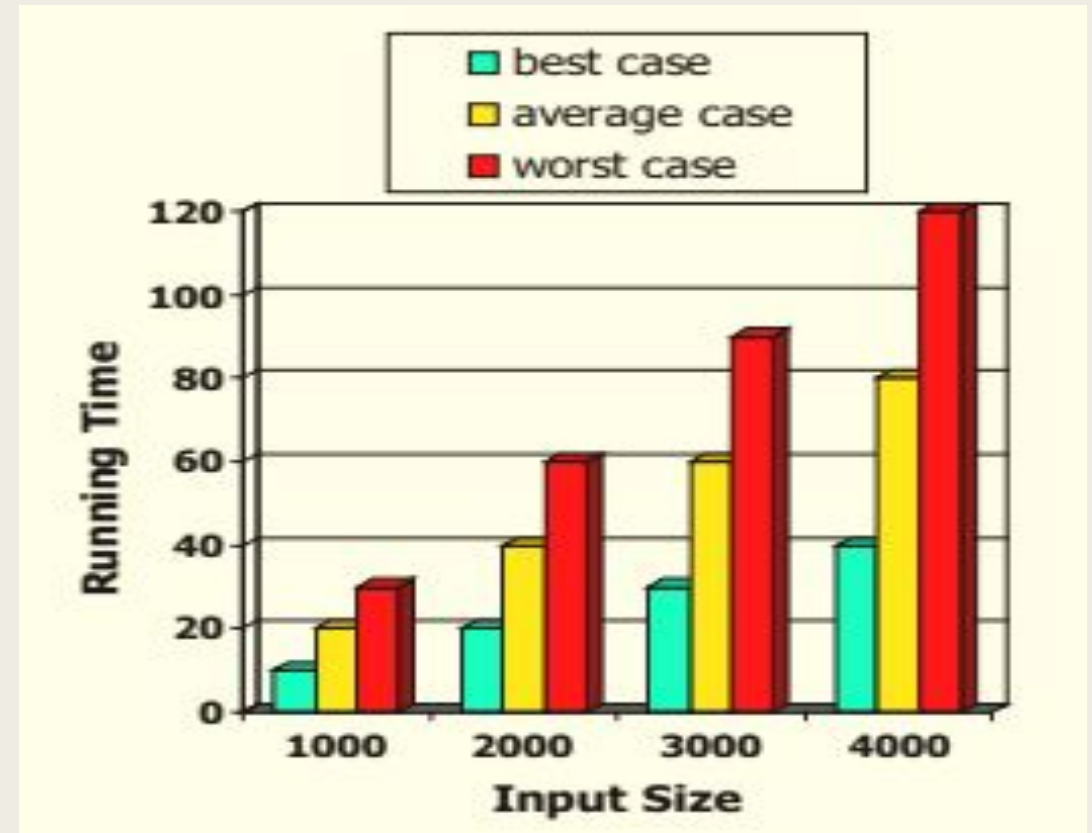


- Algorithm running time typically grows with the input size
- **Analyze running time as a function of input size**
 - $T(n)$, where n is integer expressing the input size
 - Example: n is the size of the input array sorting

How to Calculate Running time

- Even on inputs of the same size, running time can be different
 - Example: algorithm that finds the first prime number in an array by scanning it left to right
- Idea: analyze running time in the
 - best case
 - worst case
 - average case

- Best case running time is usually useless
- Average case running time is very useful but often difficult to determine
- We focus on the worst case running time
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics



Algorithm Complexity

- **Worst Case Complexity:**

- *the function defined by the maximum number of steps taken on any instance of size n*

- **Best Case Complexity:**

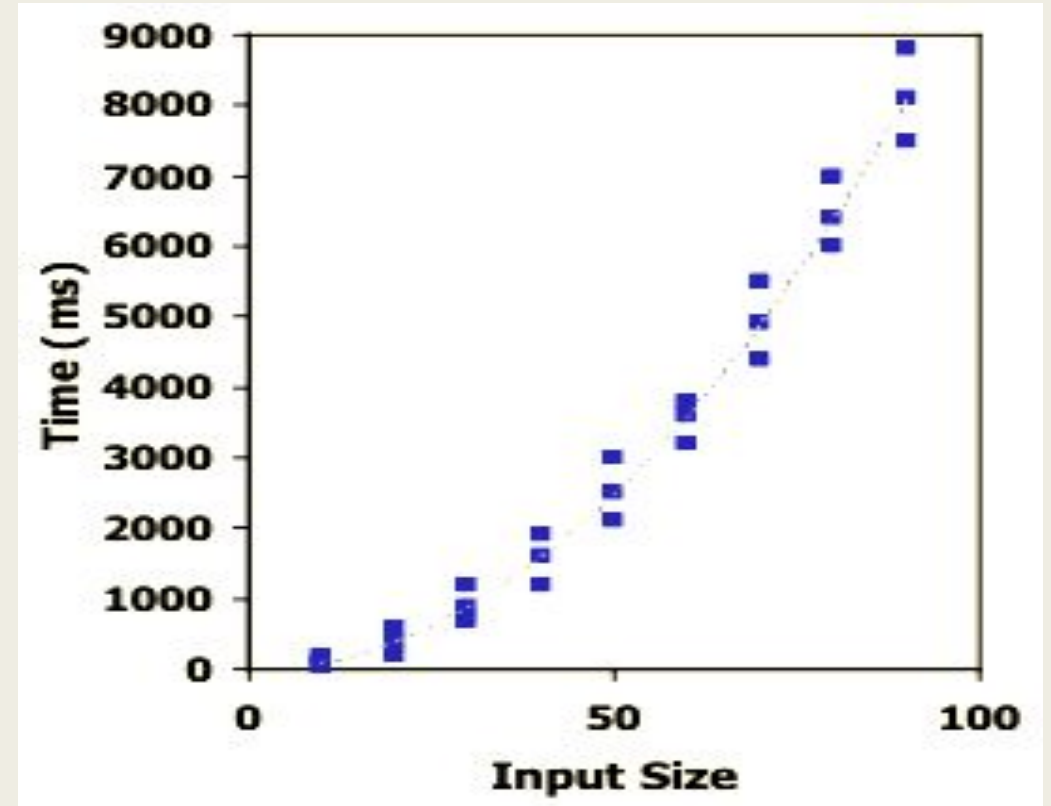
- *the function defined by the minimum number of steps taken on any instance of size n*

- **Average Case Complexity:**

- *the function defined by the average number of steps taken on any instance of size n*

Experimental Evaluation of Running Time

- Implementing algorithm
- Run the program with inputs of varying size and composition
- Plot results



Limitations of Experiments

- Experimental evaluation of running time is useful *but*
 - necessary to implement algorithm, which may be difficult
 - results may not be indicative of the running time on other inputs not included in the experiment
 - to compare two algorithms, the same hardware and software environments must be used

Theoretical Analysis of Running Time

- Uses a pseudo-code description of the algorithm instead of implementation
- Characterizes running time as a function of the input size, n
- Takes into account all possible inputs
- Allows to evaluate the speed of algorithm independent of hardware/software

Simple Example (1)

// Input: int A[N], array of N integers

// Output: Sum of all numbers in array A

```
int Sum(int A[], int N)
{
    int s=0;
    for (int i=0; i< N; i++)
        s = s + A[i];
    return s;
}
```

How should we analyse this?

Simple Example

// Input: int A[N], array of N integers
// Output: Sum of all numbers in array A

```
int Sum(int A[], int N){
```

```
    int s=0; ← ①
```

```
    for (int i=0; i< N; i++) ← ④
```

②

③

④

```
        s = s + A[i]; ← ⑤
```

⑥

⑦

```
    return s; ← ⑧
```

```
}
```

1,2,8: Once

3,4,5,6,7: Once per each iteration
of for loop, N iteration

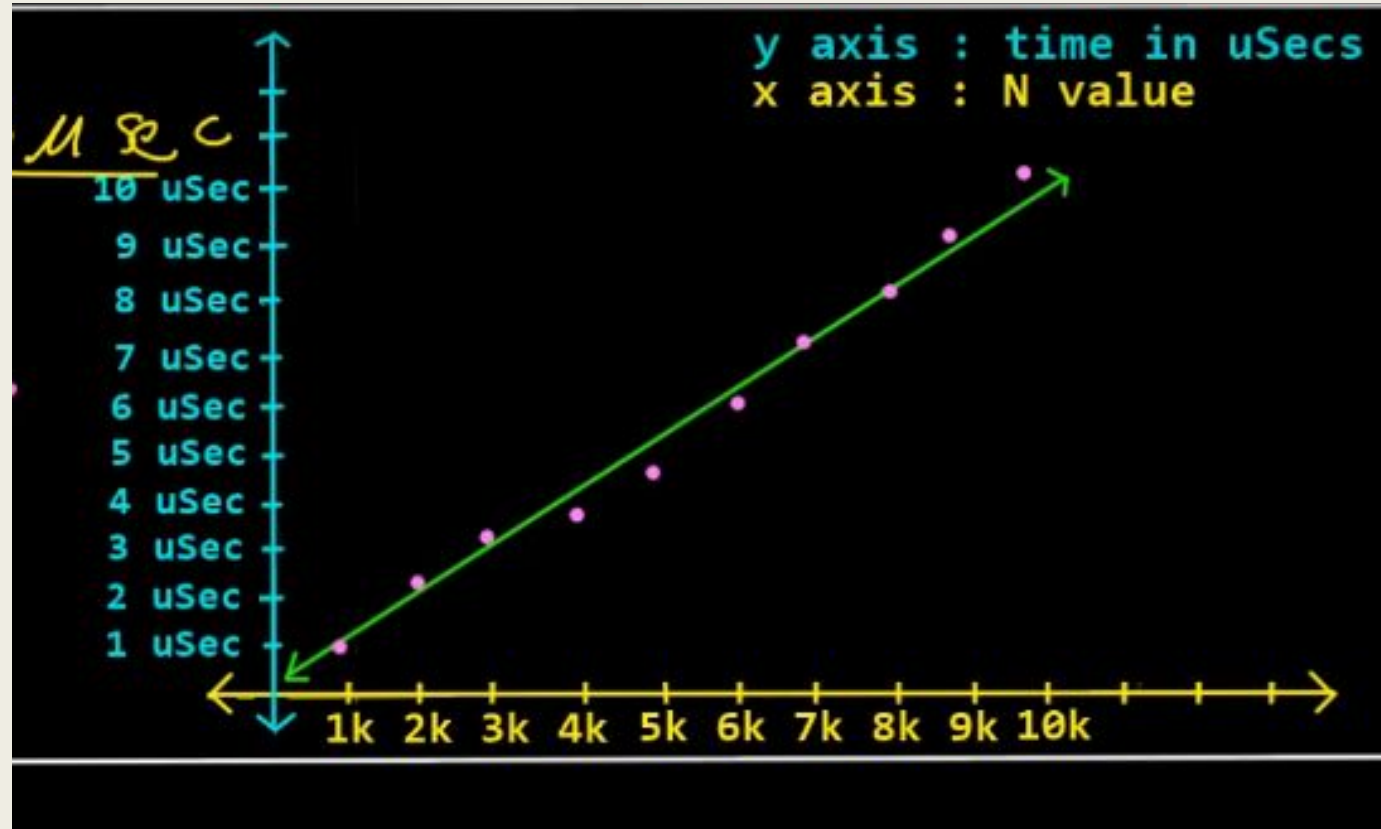
Total: $5N + 3$

The *complexity function* of the
algorithm is : $f(N) = 5N + 3$

Algorithm 1

- Problem Statement: Write an algorithm/program to find SUM of N numbers
- function sumofNumbers(N)
 - {
 - 1. sum=0 1 ms
 - 2. for(i=1 to N) 1+n ms
 - {
 - 3. sum = sum + i n ms
 - }
 - 4. print(sum) 1 ms
 - }

Input Size and Time Relationship

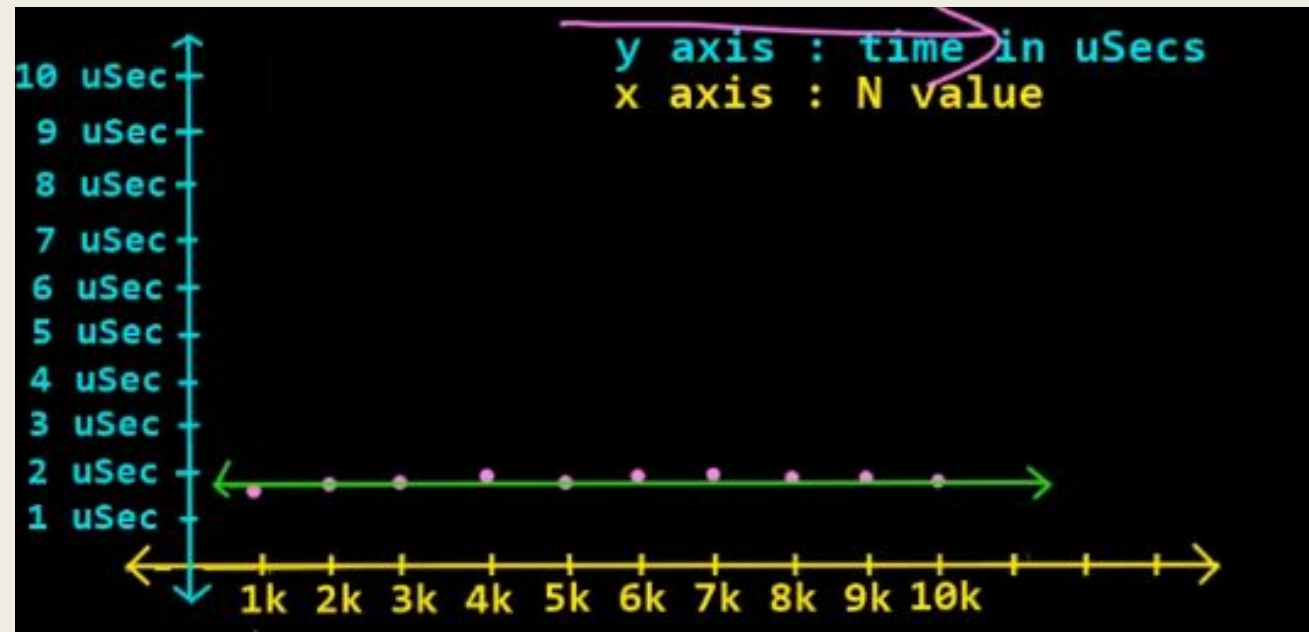


Algorithm 2

- Problem Statement: Write an algorithm/program to find SUM of N numbers

```
function sumofNumbers(N)
{
1. sum = (N*(N+1))/2    1 ms
2.print(sum)           1 ms
}
```

Input Size and Time Relationship



Standard Analysis Techniques

- Constant time statements
- Analysing Loops
- Analysing Nested Loops
- Analysing Sequence of Statements
- Analysing Conditional Statements

Constant time statements

- Simplest case: $O(1)$ time statements
- Assignment statements of simple data types
`int x = y;`
- Arithmetic operations:
`x = 5 * y + 4 - z;`
- Array referencing:
`A[j] = 5;`
- Most conditional tests:
`if (x < 12) ...`

Analyzing Loops

Any loop has two parts:

- *How many iterations are performed?*
- *How many steps per iteration?*

```
int sum = 0,j;
```

```
for (j=0; j < N; j++)
```

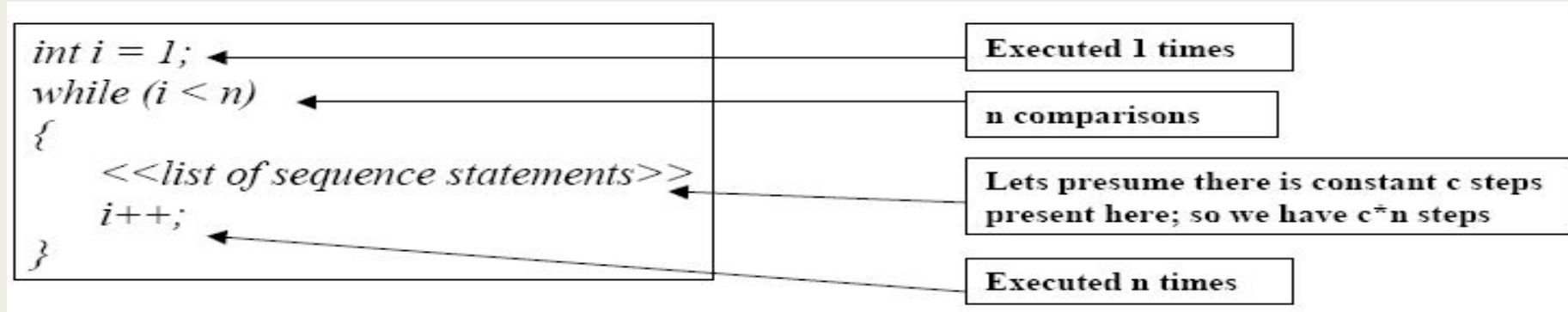
```
sum = sum +j;
```

- *Loop executes N times ($0..N-1$)*
- *$4 = O(1)$ steps per iteration*

Total time is $N + O(1) = O(N+1) = O(N)$

Analyzing Loops – Linear Loops

- Example (have a look at this code segment):



- Efficiency is proportional to the number of iterations.
- Asymptotically, efficiency is : $O(n)$

Analyzing Nested Loops[1]

- Treat just like a single loop and evaluate each level of nesting as needed:

```
int j,k;  
for (j=0; j<N; j++)  
    for (k=N; k>0; k--)  
        sum += k+j;
```

- Start with outer loop:
 - *How many iterations? N*
 - *How much time per iteration? Need to evaluate inner loop*
- Inner loop uses $O(N)$ time
- Total time is $N * O(N) = O(N*N) = O(N^2)$

Sequence of Statements

For a sequence of statements, compute their complexity functions individually and add them up

```
for (j=0; j < N; j++)  
    for (k =0; k < j; k++)  
        sum = sum + j*k;  
for (l=0; l < N; l++)  
    sum = sum -l;  
System.out.print("sum is now"+sum);
```

} $O(N^2)$
}
}
} $O(1)$

Total cost is $O(n^2) + O(n) + O(1) = O(n^2)$

Conditional Statements

- What about conditional statements such as
if (condition)
 statement1;
else
 statement2;
- where statement1 runs in $O(n)$ time and statement2 runs in $O(n^2)$ time?
- We use "worst case" complexity: among all inputs of size n , what is the maximum running time?
- The analysis for the example above is $O(n^2)$

Asymptotic Notations

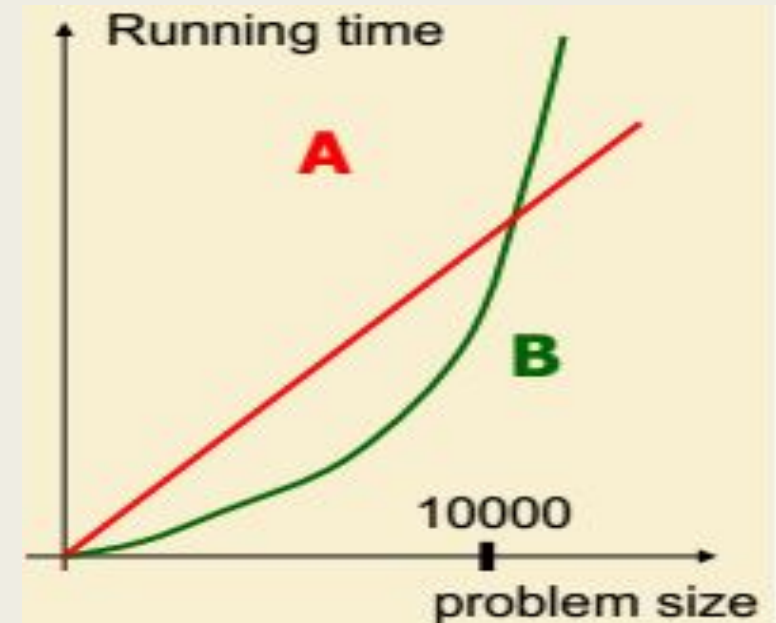
- When it comes to analyzing the complexity of any algorithm in terms of time and space, we can never provide an exact number to define the time required and the space required by the algorithm, instead we express it using some standard notations, also known as **Asymptotic Notations**
- We use three types of asymptotic notations to represent the growth of any algorithm, as input increases:
 - Big Theta (Θ)* *Tight bound(average)*
 - Big Oh(O)* *Upper bound (worst)*
 - Big Omega (Ω)* *Lower bound (best)*

Asymptotic Analysis Motivation

- big-O notation is used widely to characterize running times and space bounds
- big-O notation allows us to
 - ignore constant factors and
 - ignore lower order terms
 - focus on the main components that affect growth rate

Big- O

- We focus on the asymptotic growth using big-O notation, but practitioners do care about constant factors occasionally
 - Suppose
 - Algorithm A has running time $30000n$
 - Algorithm B has running time $3n^2$
 - Asymptotically, algorithm A is better than algorithm B



Big- O

The general step wise procedure for Big-O runtime analysis is as follows:

- Figure out what the input is and what n represents.
- Express the maximum number of operations, the algorithm performs in terms of n .
- Eliminate all excluding the highest order terms.
- Remove all the constant factors.

Examples

- $7n+2$

$7n+2$ is $O(n)$

- $3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

- $3\log n + 5$

$3 \log n + 5$ is $O(\log n)$